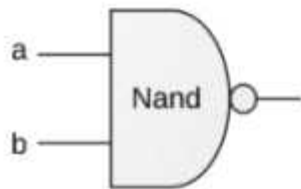


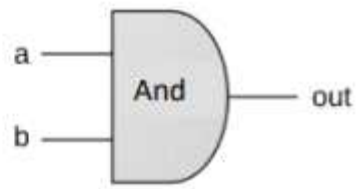
<불논리>

모든 소자는 NAND 소자만으로 구현할 수 있다.

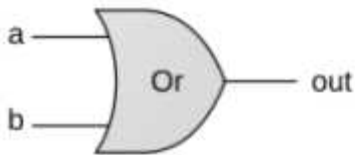
1.Elementary Gates



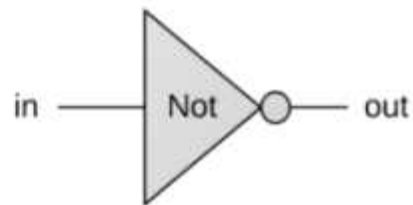
if (a==1 and b==1)
then out=0 else out=1



if (a==1 and b==1)
then out=1 else out=0

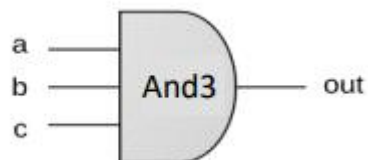


if (a==1 or b==1)
then out=1 else out=0

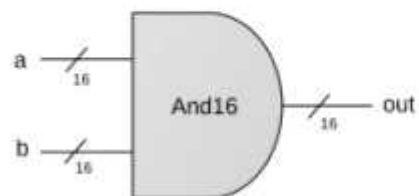


if (in==0)
then out=1 else out=0

2.Composite Gates (Mux, Adder...)



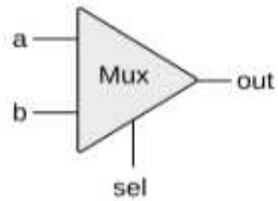
And16



Example:

```
a = 1 0 1 0 1 0 1 1 0 1 0 1 1 1 0 0
b = 0 0 1 0 1 1 0 1 0 0 1 0 1 0 1 0
out = 0 0 1 0 1 0 0 1 0 0 0 0 0 1 0 0
```

Multiplexor

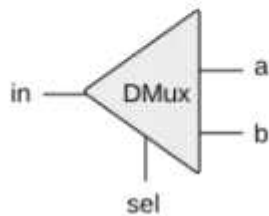


```
if (sel == 0)
    out = a
else
    out = b
```

a	b	sel	out
0	0	0	0
0	1	0	0
1	0	0	1
1	1	0	1
0	0	1	0
0	1	1	1
1	0	1	0
1	1	1	1

sel	out
0	a
1	b

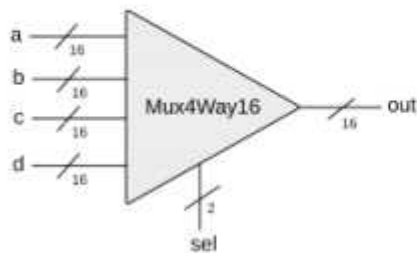
Demultiplexor



```
if (sel == 0)
    {a, b} = {in, 0}
else
    {a, b} = {0, in}
```

in	sel	a	b
0	0	0	0
0	1	0	0
1	0	1	0
1	1	0	1

16-bit, 4-way multiplexor



sel[1]	sel[0]	out
0	0	a
0	1	b
1	0	c
1	1	d

3. 부록 A 내용 정리

*HDL : Hardware Description Language

1). 규칙

- *코드를 작성하고 파일확장자를 .hdl로 한다.
- *칩구조는 헤더와 본문으로 구성된다.
헤더는 칩의 인터페이스를 정의, 본문은 구현
- *HDL은 대소문자를 구분한다. HDL 키워드는 대문자이다.
- *명명 규칙 : 칩과 핀의 이름은 맨 앞글자가 숫자가 아닌 문자와 숫자로 이루어진 문자열.
보통 칩이름은 대문자/ 핀이름은 소문자로 시작.
- *공백문자/개행문자/주석은 무시한다.

*주석 종류

```
//라인 끝까지 주석  
/* 달을 때까지 주석 */  
/** API 문서 주석*/
```

2). 하드웨어 시뮬레이터에 칩 로드

- *프로그램에서 'load file'을 통해 HDL 파일 연다.
- *로드시 내부 부품으로 정의된 모든 칩이름에 대해 .hdl 파일이 로드된다.
(하위 계층 칩들도 같이 로드된다)
- *시뮬레이터의 builtIn 디렉토리에 대부분의 칩 파일이 있다. 따라서 사용자가 현재 디렉토리에 있는 하위 칩 파일을 옮기더라도 프로그램에 내장된 칩정보를 끊어올 수 있다.

3). 칩 헤더(인터페이스)

```
/*  
CHIP chip name {  
    IN input pin name, input pin name ...;  
    OUT output pin name, output pin name...;  
}  
*/
```

- *CHIP은 칩의 이름을 선언한다.
- *IN은 입력핀의 이름을 선언한다. 입력핀의 목록은 세미콜론으로 끝난다.
- *OUT은 출력핀의 이름을 선언한다. 출력핀의 목록은 세미콜론으로 끝난다.
- *입력 및 출력 핀은 기본으로 1비트로 가정한다.
- *멀티비트 버스는 pin name[w] 표기법으로 선언 할 수 있다.
이런 경우에는 name의 인덱스는 0~w-1이고 인덱스0는 최하위 비트 참조한다.

4). 칩 본문(구현)

```
/*  
PARTS:  
internal chip part;  
internal chip part;  
...  
internal chip part;  
*/
```

*internal chip part 명령문은 한개의 내부칩과 연결을
chip name (connection, ..., connection)
part의 pin name = chip의 pin name
으로 정의한다.

*내부 핀을 이용한다. 변수같은 것이다.

```
/*  
Part1(out=v)    // Part1의 out에 v가 연결됨.  
Part2(in=v)      // v는 Part2의 in에 연결됨.  
Part3(a=v, b=v) // v는 Part3의 a와 b에 동시에 연결됨.  
*/
```

5). 버스

*멀티비트 버스의 입출력 핀에서 특정 인덱스만 따로 연결할 때는
x[i...j]=v 방식으로 한다. (i에서 j 인덱스 비트만 연결)

6). 내장형 칩

내장형 칩은 자바같은 프로그래밍 언어코드로 구현되어 HDL인터페이스 아래에서 동작한다.

```
/** 16비트 다중화기  
sel=0이면 out=a, 아니면 out=b.  
이 칩은 외부 자바 클래스로 구현한 내장형 칩이다. */  
CHIP Mux16 {  
    IN a[16], a[16], sel;  
    OUT out[16];  
    BUILTIN Mux; // builtIn/Mux.class를 참조함  
                // 이 클래스는 Mux.hdl과  
                // Mux16.hdl 내장형 칩을 둘 다 구현함  
}
```

*BUILTIN 다음의 식별자는 칩을 구현한 프로그램 이름.

7). 순차 칩

컴퓨터 칩들은 Combinational(조합)/Sequential(순차) 방식이다.

조합은 동작이 즉각적이다. 조합칩의 입력 핀 값을 바꾸면 바로 값이 바뀐다.

순차는 클록에 좌우된다.

입력이 변하면 클록에 따라 다음 시간단위 시작시 칩의 출력이 변경된다.

8). 클록

시뮬레이터는 Tick과 Tock이라 불리는 두 연산으로 시간의 흐름을 모델링 한다.

Tick 신호는 시간 단위의 첫 단계를 끝내고 두 번째 단계를 시작.

Tock 신호는 다음 시간 단위의 첫 번째 단계를 시작.

*시뮬레이터의 사용자나 테스트 스크립트는 마음대로 틱과 톡을 발생시켜 시간 단위를 연속적으로 진행 시킬 수 있다.

*클록을 제어하는 방법은 수동/스크립트 이용방법 2가지 이다.

수동식은 시뮬레이터의 GUI의 클록 모양 버튼 이용.

테스트 스크립트는 repeat n {tick, tock, output}; 명령어 사용

시간 단위 n만큼 클록을 진행시키면서 어떤 값들을 출력하도록 시뮬레이터에게 지시하는 명령이다.

9). 클록 칩과 핀

내장형 핀은 CLOCKED pin, pin,... pin;을 통해서 클록 의존성 선언한다.

여기서 pin은 칩 헤더에서 선언된 입출력핀이다.

10). 피드백 루프

부품의 하나의 출력이 같은 부품의 입력에 직간접적으로 영향을 줄 때 피드백 루프.

시뮬레이터는 칩을 로드할 때, 피드백 루프가 있는지 검사하고, 각 루프마다 클록 핀을 통과하는지 체크한다. 클록핀을 통과하지 않으면 에러 메세지 발생시킨다.

11). 칩 연산의 시각화

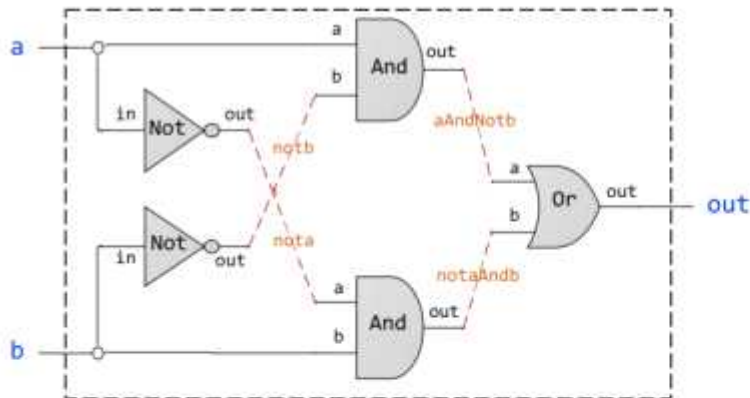
내장형 칩은 GUI 기능이 있다.

키보드칩, 스크린칩, ROM32k칩, 주소레지스터, 데이터 레지스터, ALU 등이 있다.

**불논리 연산 방식을 이용하면 빠르게 식을 구해 회로구성가능하다.

진리표만 알면 회로를 식으로 구할 수 있다.

12) 예시



```

/** out = {a And Not(b)) Or (Not(a) And b)) */
CHIP Xor {
  IN a, b;
  OUT out;
  PARTS:
    Not (in=a, out=nota);
    Not (in=b, out=notb);
    And (a=a, b=notb, out=aAndNotb);
    And (a=nota, b=b, out=notaAndb);
    Or (a=aAndNotb, b=notaAndb, out=out);
}

```



```

/** Chips set (APIs): */
...
Not (in=, out= );
And (a=, b=, out= );
Or (a=, b=, out= );
Xor (a=, b=, out= );
...

```

```

CHIP Nandd {
  IN a,b;
  OUT out;

  PARTS:
    And (a=a, b=b, out=nodeA);
    Not (in=nodeA, out =out);
}

```

```

CHIP Xorr {
  IN a, b;
  OUT out;

  PARTS:
    Not(in=a, out=nodeA);
    Not(in=b, out=nodeB);
    And(a=nodeA, b=b, out=nodeC);
    And(b=nodeB, a=a, out=nodeD);
    Or(a=nodeC, b=nodeD, out=out);
}

```

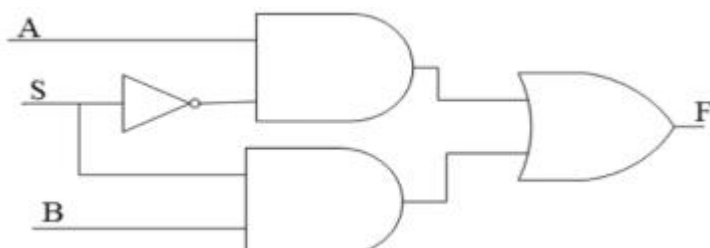
```
CHIP And3 {
  IN a,b,c;
  OUT out;

  PARTS:
  And(a=a,b=b, out=nodeA);
  And(a=nodeA, b=c, out=out);
}
```

```
CHIP And16 {
  IN a[16], b[16];
  OUT out[16];

  PARTS:
  And(a=a[0], b=b[0], out=out[0]);
  And(a=a[1], b=b[1], out=out[1]);
  And(a=a[2], b=b[2], out=out[2]);
  And(a=a[3], b=b[3], out=out[3]);
  And(a=a[4], b=b[4], out=out[4]);
  And(a=a[5], b=b[5], out=out[5]);
  And(a=a[6], b=b[6], out=out[6]);
  And(a=a[7], b=b[7], out=out[7]);
  And(a=a[8], b=b[8], out=out[8]);
  And(a=a[9], b=b[9], out=out[9]);
  And(a=a[10], b=b[10], out=out[10]);
  And(a=a[11], b=b[11], out=out[11]);
  And(a=a[12], b=b[12], out=out[12]);
  And(a=a[13], b=b[13], out=out[13]);
  And(a=a[14], b=b[14], out=out[14]);
  And(a=a[15], b=b[15], out=out[15]);
}
```

*Mux



```

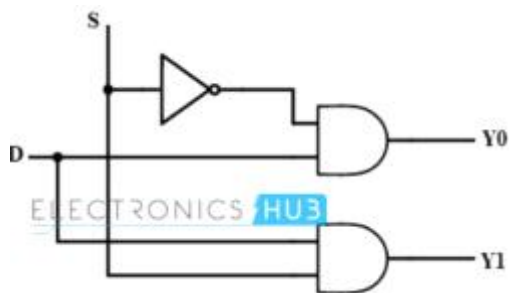
CHIP Muxx {
  IN a, b, sel;
  OUT out;

  PARTS:
    Not(in=sel,out=selN);
    And(a=a,b=selN,out=nodeA);
    And(a=sel, b=b, out=nodeB);
    Or(a=nodeA, b=nodeB, out=out);

}

```

*DMUX



```

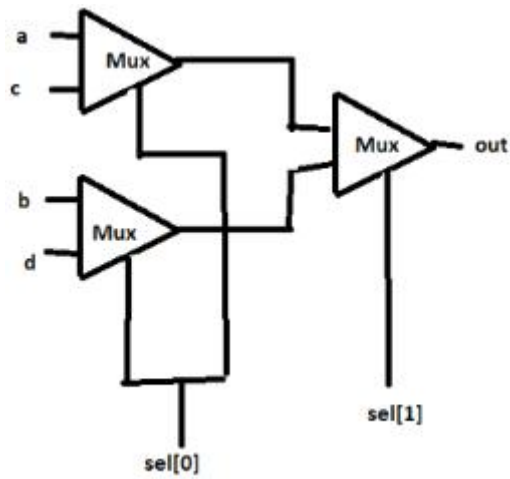
CHIP Dmux {
  IN in, sel;
  OUT a, b;

  PARTS:
    Not(in=sel, out=selN);
    And(a=in, b=selN, out=a);
    And(a=sel, b=in, out=b);

}

```


*Mux4Way



```
CHIP Mux4Way {
  IN a,b,c,d,sel[2];
  OUT out;
```

PARTS:

```
Muxx(a=a,b=b,sel=sel[0],out=nodeA);
```

```
Muxx(a=c,b=d,sel=sel[0],out=nodeB);
```

```
Muxx(a=nodeA,b=nodeB, sel=sel[1],out=out);
```

```
}
```

```
CHIP Mux16 {  
  IN a[16], b[16], sel;  
  OUT out[16];
```

```
  PARTS:
```

```
  Mux(a=a[0],b=b[0], sel=sel, out=out[0]);  
  Mux(a=a[1],b=b[1], sel=sel, out=out[1]);  
  Mux(a=a[2],b=b[2], sel=sel, out=out[2]);  
  Mux(a=a[3],b=b[3], sel=sel, out=out[3]);  
  Mux(a=a[4],b=b[4], sel=sel, out=out[4]);  
  Mux(a=a[5],b=b[5], sel=sel, out=out[5]);  
  Mux(a=a[6],b=b[6], sel=sel, out=out[6]);  
  Mux(a=a[7],b=b[7], sel=sel, out=out[7]);  
  Mux(a=a[8],b=b[8], sel=sel, out=out[8]);  
  Mux(a=a[9],b=b[9], sel=sel, out=out[9]);  
  Mux(a=a[10],b=b[10], sel=sel, out=out[10]);  
  Mux(a=a[11],b=b[11], sel=sel, out=out[11]);  
  Mux(a=a[12],b=b[12], sel=sel, out=out[12]);  
  Mux(a=a[13],b=b[13], sel=sel, out=out[13]);  
  Mux(a=a[14],b=b[14], sel=sel, out=out[14]);  
  Mux(a=a[15],b=b[15], sel=sel, out=out[15]);
```

```
}
```

*Mux4Way16

Mux16과 Mux4Way 참조하면된다.